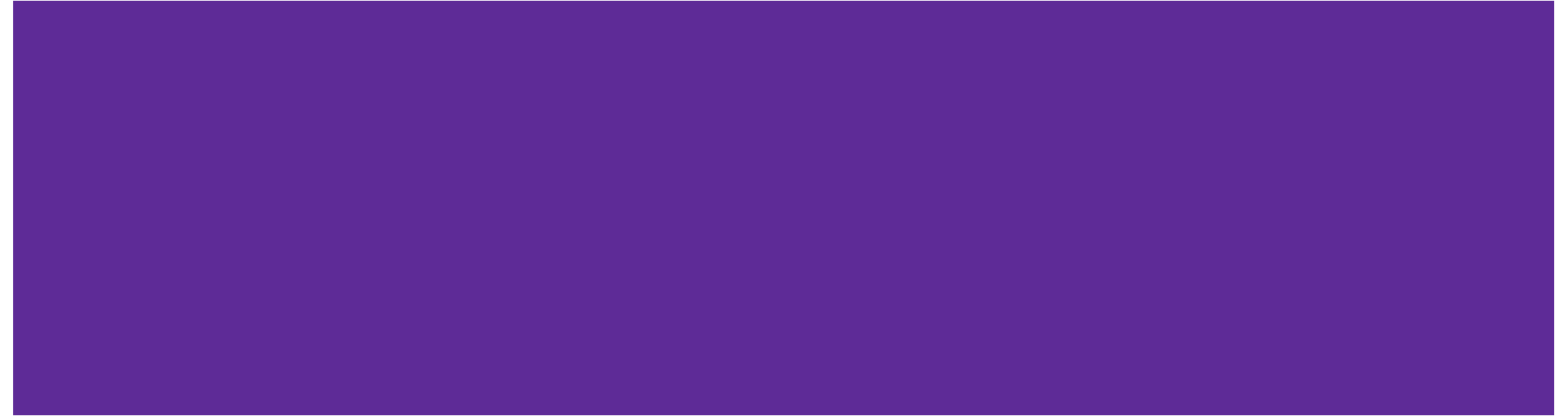


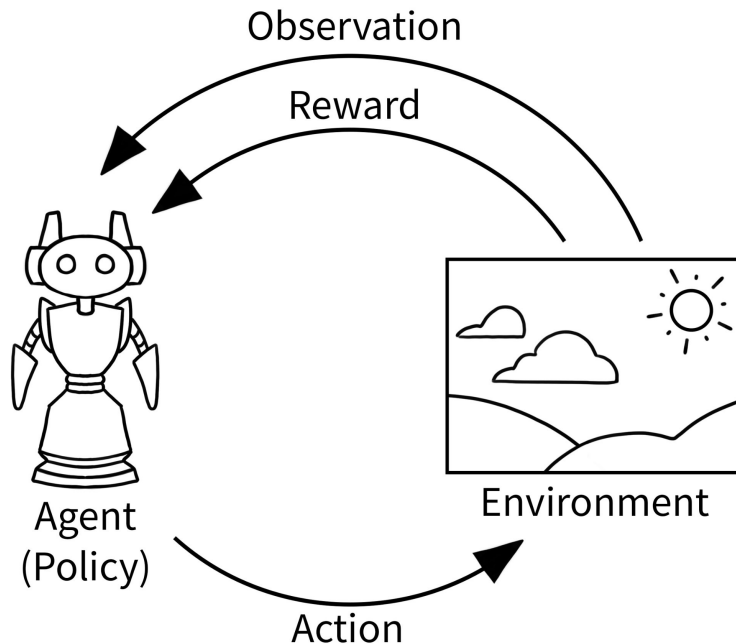
Intelligent Systems

Gymnasium tutorial



What is Gymnasium?

We want to program
(or train) an agent.



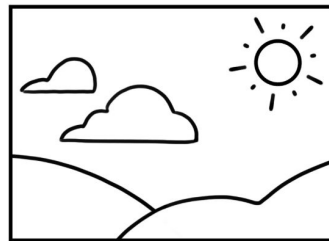
But where do
we test it?

Real world?

Simulation?

What is Gymnasium?

- Gymnasium is an open source Python library that provides many different environments to train agents.
- Standard API to communicate between learning algorithms and environments.
- Agents can easily be transferred between different environments.
- It makes published research more easily reproducible.
- When developing (publishing) your own environment, it is advisable to follow the Gymnasium interface for an easier adaptation by the research community.



Environment

History

2015 - OpenAI (San Francisco, California) is founded.

2016 - OpenAI Gym is released as a tool for Reinforcement Learning development.

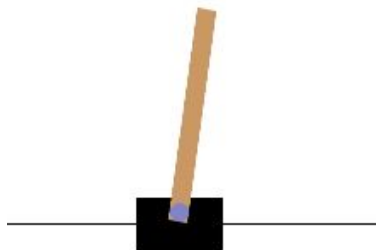
2020 - Maintenance of Gym largely decreased.

2022 - OpenAI handed over the maintenance of Gym to an outside team
(The Farama Foundation).

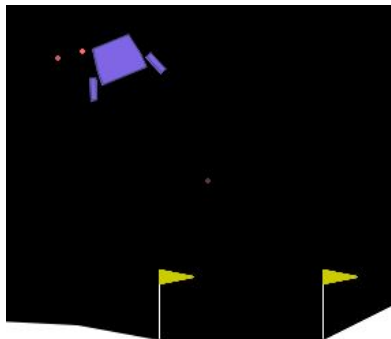
The name is changed to Gymnasium.

2024 - Version 1.0.0 of Gymnasium is released.

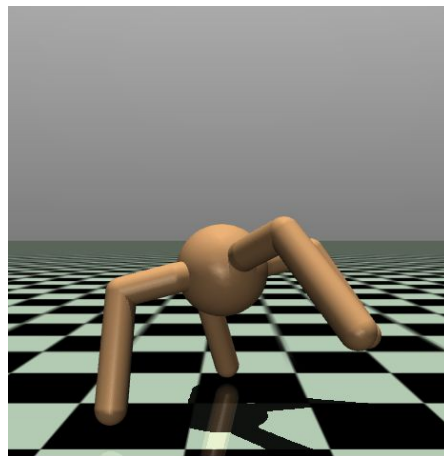
Built-in environments



Classic control



Box2D



MuJoCo



Atari

Installation

1. Install Python (with PIP package manager).
2. Create a new Python virtual environment (give it a name).

```
python -m venv <name>
```

3. Activate the environment.

```
source ./<name>/bin/activate
```

4. Update PIP (for just in case).

```
python -m pip install --upgrade pip
```

5. Install Gymnasium.

```
python -m pip install gymnasium
```

First steps

```
import gymnasium as gym

env = gym.make("CartPole-v1", render_mode="human")
observation, info = env.reset()

episode_over = False
total_reward = 0

while not episode_over:
    action = env.action_space.sample() # Random action for now
    observation, reward, terminated, truncated, info = env.step(action)

    total_reward += reward
    episode_over = terminated or truncated

env.close()
```

What does it mean?

- `env = gym.make("CartPole-v1", render_mode="human")`

Create the chosen environment (Cart-Pole v1 in this case).

Choose render mode “human” to render the environment or None (default) for maximum speed.

- `env.reset()`

Reset the environment to its initial state. Must be called before first use.

- `action = env.action_space.sample()`

Returns a random action (usually we want the agent to choose actions in a smarter way).

- `env.step(action)`

Executes the given action.

- `env.close()`

Destroys the environment.

Observations

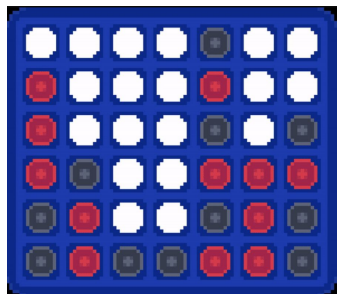
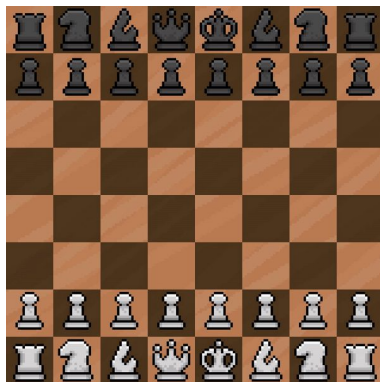
After executing an action, the agent gets a feedback from the environment:

```
observation, reward, terminated, truncated, info = env.step(action)
```

- Observation - the new state of the environment.
- Reward - how good (or bad) the executed action is.
- Terminated - True, if the goal has been reached or if the episode failed.
- Truncated - True, if the episode has been cut short before success/fail.
- Info - Additional info (if any) about the current state of the environment.

PettingZoo

- The core Gymnasium library is a single-agent framework.
- PettingZoo (<https://pettingzoo.farama.org/>) is a multi-agent version of Gymnasium.
- Many two-player games are available out of the box.



Example of use

```
from pettingzoo.butterfly import knights_archers_zombies_v10

env = knights_archers_zombies_v10.env(render_mode="human")
env.reset(seed=42)

for agent in env.agent_iter():
    observation, reward, termination, truncation, info = env.last()

    if termination or truncation:
        action = None

    else:

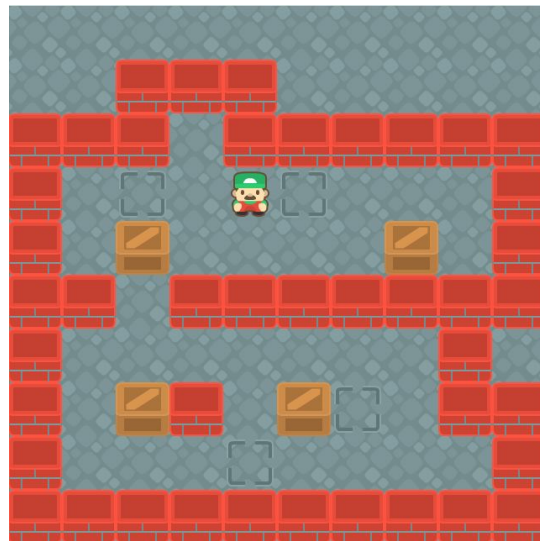
        # this is where you would insert your policy
        action = env.action_space(agent).sample()
    env.step(action)
```

Custom environments



Sokoban

- We will use a few custom Gymnasium environments.
- Sokoban - a popular puzzle game from 1981.
- Actions: UP (0), RIGHT (1), DOWN (2), LEFT (3).
- Observation: 10 x 10 grid.
- Goal: Every crate is at a designated goal position.
- Limitations: Crates can only be pushed one at a time.
- Reward: 1 if game finished, 0 otherwise.



Sokoban

Actions: An integer

Integer	Meaning
0	UP
1	RIGHT
2	DOWN
3	LEFT

Observations: A 2D numpy array of integers.

Integer	Tile	Meaning
0		Floor
1		Wall
2		Crate
3		Goal
4		Crate on goal
5		Player

Sokoban

Options: a dictionary of options can be given at environment creation.

```
map = np.array( # The array can be of any dimensions greater than or equal to 2 × 2.
```

```
[
    [0,0,0,0,0,0,1,0,0],
    [2,0,1,1,1,3,1,0,0],
    [0,1,2,0,1,0,0,0,0],
    [0,1,0,0,1,0,0,0,0],
    [0,0,2,0,0,0,0,0,0],
    [0,5,0,0,2,0,2,0,0],
    [0,0,0,0,0,0,0,0,0],

```

```
])
)
```

```
map = 151 # It can also be an integer between 0 and 999, representing one of 1000 built-in maps.
```

```
map = None # Or None to randomly choose one of 1000 maps (default behavior).
```

```
env = gym.make('famnit_gym/Sokoban-v1', render_mode='human', options={'map_template': map, 'scale': 0.75})
```

Sokoban

Wrappers: Extend the basic functionality.

```
from famnit_gym.wrappers.sokoban import Keyboard, Insights

# Wrap the environment into the Keyboard wrapper to allow a human to play the game.
env = gym.make('famnit_gym/Sokoban-v1', render_mode='human')
env = Keyboard(env)

# Wrap the environment into the Insights wrapper to get more information in the info structure.
env = Insights(env)

info = {
    'steps': 23,                                # The number of steps made this far (inherited from the base info).
    'player': [x, y],                            # The location of the player.
    'crates': [[x0, y0], [x1, y1], ...],          # The locations of the crates.
    'goals': [[x0, y0], [x1, y1], ...],          # The locations of the goals.
    'finished': [[x0, y0], [x1, y1], ...],        # The locations of the crates on goal positions.
    'actions': {
        'moving': [0, ..., 3],                    # Actions currently effective for moving.
        'pushing': [0, ..., 3]                   # Actions currently effective for pushing.
    }
}
```

Installation

1. Create a new Python virtual environment (give it a name).

```
python -m venv <name>
```

2. Activate the environment.

```
source ./<name>/bin/activate
```

3. Update PIP (for just in case).

```
python -m pip install --upgrade pip
```

4. Install the famnit-gym package:

```
pip install famnit_gym@git+https://github.com/DomenSoberlFamnit/famnit-gym
```

5. Try, if it works:

```
python -m famnit_gym.examples.sokoban_keyboard
```

A minimal example

```
import gymnasium as gym
import famnit_gym

# Create and reset the environment.
env = gym.make('famnit_gym/Sokoban-v1', render_mode='human')
observation, info = env.reset()

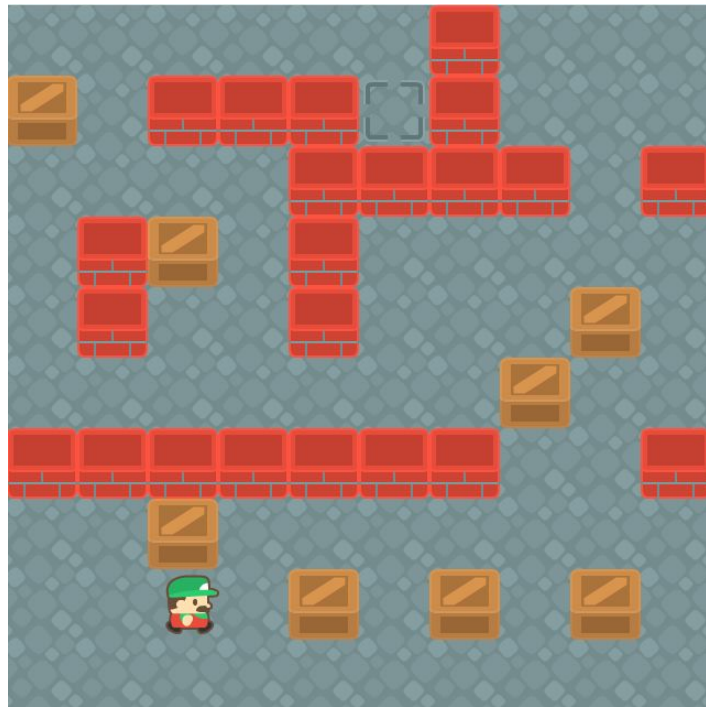
# Execute random actions.
done = False

while not done:
    action = env.action_space.sample()
    _, _, terminated, truncated, _ = env.step(action)
    done = terminated or truncated
```

Exercise

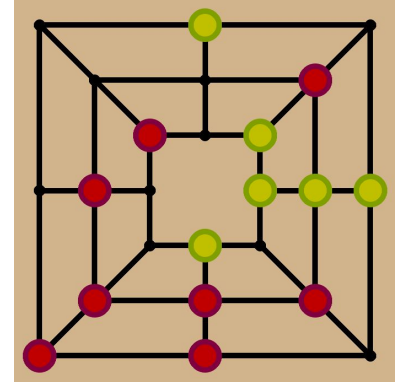
- Create the map as shown in the image.
- Implement the A* search algorithm to find a path to the goal location.
- Execute the found solution in our Gymnasium Sokoban environment.
- Question: Is the Manhattan distance between the player and the goal location an admissible heuristic? If yes, what does it mean?
- You may try out the solution of this task by running:

```
python -m famnit_gym.examples.sokoban_pathfind
```



Nine men's morris (Mill)

- A very old game, dating back at least to the Roman empire.
- Known all over the world - the rules might vary.
- Each player has 9 pieces that they place / move on the board.
- Three pieces in a line form a mill → capture one opponent's piece.
- Three phases: placing, moving, flying.
- The player left with the less than 3 pieces loses.
- The game was *solved* in 1998: a perfect strategy is possible that never loses.



Nine men's morris (Mill)

Actions: [src, dst, capture]

Variable	Meaning
src	The source position (1 - 24) of the piece or 0 when placing.
dst	The destination position (1 - 24) of the piece.
capture	The position (1 - 24) from which the opponent's piece is captured or 0 if none captured.

- Legal actions are listed in `info[ 'legal_actions' ]`.
- Passing an illegal action (or None) to the environment results in executing a random legal action.

Nine men's morris (Mill)

Observations: array of 24 integers, representing the state of each position.

Value	Meaning
0	The position is empty.
1	The position is occupied by player 1.
2	The position is occupied by player 2.

Additional information is given in the `info` structure:

Number of moves, the current agent, the phase (placing, moving, flying, lost) of the agent.

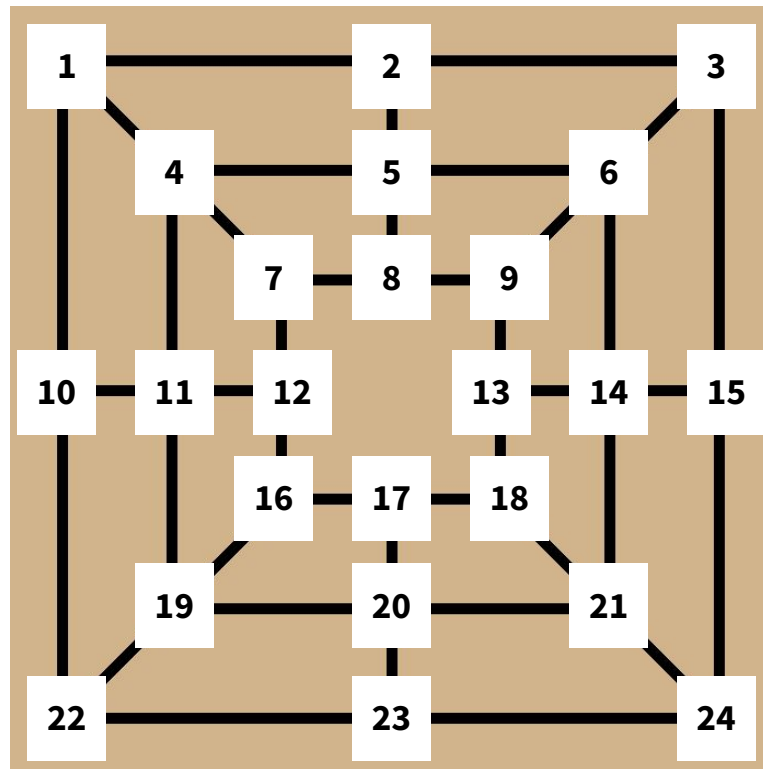
Nine men's morris (Mill)

The arrangement of the board

Careful:

the indices in the *observation* array are zero-based.

```
piece = observations[position - 1]
```



Nine men's morris (Mill)

Transition model

- To implement an offline search, you need a transition model.
- The model for this game is rather complex.
- You don't have to write your own, you can use an existing one:

```
from famnit_gym.envs import mill  
  
model = mill.transition_model(env)
```

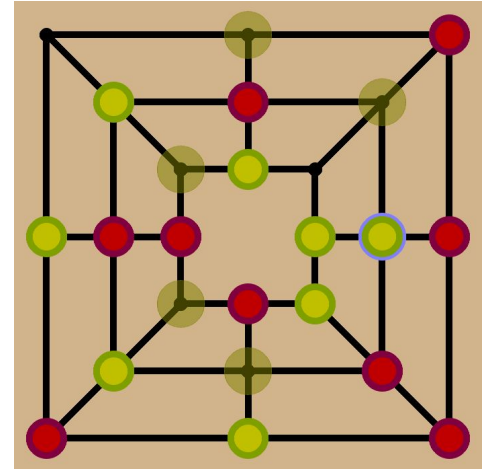
See an example [here](#) for more information.

Nine men's morris (Mill)

User interaction

- A two-player game is most interesting when a human can play with AI.
- There is a wrapper [UserInteraction](#) that allows capturing the user input through mouse and keyboard.
- It also allows to put color markings on the board's positions (1 - 24).

See an example [here](#) for more information.



Famniti-gym

Installation:

```
pip install famniti_gym@git+https://github.com/DomenSoberlFamniti/famniti-gym
```

GitHub page:

<https://github.com/DomenSoberlFamniti/famniti-gym/>

Examples:

https://github.com/DomenSoberlFamniti/famniti-gym/tree/main/famniti_gym/examples

Questions & bug reports:

A dedicated forum the e-classroom (please, no e-mails).